

PYDANTIC ⇒ DATA VALIDATION, TYPE VALIDATION

Why → to follow Strict Schema.
(As Python Dynamic Typing, one variable can store diff. type)

* One possible way is Type Hinting.

```
def func (name: str, age: int)
```

Not very strong doesn't give error.

insert - (— , —) still it passed "hinty"
↳ would work,

* No Strict enforcing in Python.

① * Python doesn't have Type Validation to achieve this manually do type validation at multiple func not Scalable.

② Data Validation

Every argument, variable need to manually write Data and Type Validation

age → can't be negative.

* Pydantic exactly solves this.

① Define a Pydantic Model ^{↳ class} that represents ideal Schema of Data.
(expected fields, type, any Validation Constraints)

② Instantiate the model with raw input data (usually a dic. or JSON).
→ Pydantic will automatically validate data and coerce it into correct Python type (if possible)
→ if doesn't meet, Pydantic raises a Validation Error

Have a Pydantic object. (Validated)

③ Pass the Validated Model Object to functions or use it throughout your codebase.

— ensure every part of program works with clean, type-safe and logically valid Data.

```
from pydantic import BaseModel
```

```
class Patient (BaseModel):
```

```
    name: str
```

```
    age: int
```

```
patient_info = {"name": —, "age": 30}
```

```
patient1 = Patient (**patient_info)
```

```
def update_patient_data (patient: Patient)
```

Can be used.

* Also Pydantic smart enough to convert '30' to 30

class Patient(BaseModel):

name: str

age: int

weight: float

married: bool = False

allergies: List[str]

contact_details: Dict[str, str]

Patient.info = { 'name': 'nitish', 'age': 30,
'weight': 75.2, 'married': True,
'allergies': ['pollen', 'dust'],
'contact_details': { 'email': 'abc@gmail.com',
'phone': '2354767810' } }

Patient1 = Patient(**Patient.info)

if list
validation of only
list.

List[str]
None Validation
of List of
strings.

** By Default, all are required fields.
→ Optional field.

allergies: Optional[List[str]] = None.
give Default value.

* In Pydantic, two things determine
a field's Behavior.

Type Annotation
(str, str | None)

Can Value be
None

Default Value
(= ..., = None,
= "x")

is field
required.

	Required	Can be None
name: str	✓	✗
name: str = "Gaut"	✗	✗
name: str None (Optional[str])	✓	✓
name: str None = None (Optional[str] = None)	✗	✓

* Pydantic Data Validation.
↳ give custom data type.

email: EmailStr, Auto validate

linkedin_url: AnyUrl, valid URL

Custom Data Validation → use Field

Also used to
attach Metadata

Weight Can't be Negative.
Weight: float = Field(gt=0)
Age: int = Field(gt=0, lt=120)
Name: str = Field(max_length=50)

like in

Fast API
Documentation

⇒ name: Annotated[str, Field(max_length=50,
title='Name of Patient',
description='Give name less than 50 char',
example=['Nitish', 'Amit'])]

* if pydantic Auto
Conversion

Can be skipped.

weight: Annotated[_, Field(str_to_int)]

* Patient model
Checks if Patient
is HOSP employee,
ICICI employee
to give Discount

→ a type of Custom Validation use Field Validator

class Patient

@field.validator('email')

@classmethod

def email_validator(cls, value):

valid_domain = ['hotte.com', 'icici.com']

abc@gmail.com

domain_name = value.split('@')[-1]

if domain_name not in valid_domain:

raise ValueError('Not a Valid Domain')

return value

if Patient details. if age > 60
then emergency phone no must be

if not, not create Patient.

Can't use Field Validator (only on single field validation)

* Model Validation
(Combining Multiple fields data validation)

@model_validator (mode = 'after')

def validate_emergency_contact(cls, model):

if model.age > 60 and 'emergency' not in model.contact_details

raise ValueError('Patic —')

return model

abc@gmail.com

Value error,
Not a Valid
Domain.

* Any Transformation
use field validator

(mode = 'after')

default,
after Type Conversion

if (mode = 'before')

Before Type Conversion

model_validator

Computed field
Want one more field like BMI → using User given weight, height

@ Computed field
@ property
def calculate_bmi(self) → float
bmi = self.weight / (self.height ** 2)
return bmi
** name this as attribute name.
} so here, new attribute in calculate_bmi

* nested model (using one model as field inside other Model).

from pydantic import BaseModel

class Address(BaseModel):

city: str

state: str

pin: str

class Patient(BaseModel):

name: str

gender: str

age: int

address: Address

address_dict = { }

address1 = Address(**address_dict)

patient_dict = { , 'address': address1 }

patient1 = Patient(**patient_dict)

print(patient1)

name = ' ', address = Address()

* existing Pydantic Models as Python Dict or JSON (export).

temp = patient1.model_dump() ~> Convert to python Dict.

= patient1.model_dump_json() ~> to JSON

, On Control field (include = ['name'])

(exclude = ['gender'])

(exclude = { 'address': ['state'] })

(exclude_unset = True) - if during object creation, what is not set, is not converted